

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
Asignatura:	Pruebas de Software				
Título de la práctica:	Pruebas de Rendimiento y Seguridad				
Número de la práctica:	09	Año lectivo:	2026	Nro. semestre:	A
Fecha de presentación:	2026-07-23	Hora de presentación:	11:59 pm		
Integrante(s): - Arce Mayhua, Leonardo Ruben - Jara Mamani, Mariel Alisson - Quispe Condori, Álvaro Raúl - Gallegos, Anette				NOTA:	Nota colocada por el docente
Docente: Msc. Robert Edison Arisaca Mamani					

SOLUCIÓN Y RESULTADOS
1. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1.1. EJERCICIO RESUELTO POR EL DOCENTE 1.1.1. Desarrollo y Pruebas de Rendimiento de una API REST en Python utilizando Flask 1.1.1.1. Implementación de la API REST En esta sección se analiza la API REST desarrollada durante la sesión guiada utilizando el framework Flask de Python para la administración de un catálogo básico de productos. Para el desarrollo se configuró un entorno virtual en Python, donde se instalaron las dependencias necesarias: • Flask: utilizado para la creación del servidor REST. • Requests: utilizado para realizar pruebas de consumo de la API mediante Python. La API fue implementada en el archivo app.py, donde se definieron los endpoints necesarios para realizar operaciones CRUD sobre los productos: • GET /productos: permite obtener la lista completa de productos. • POST /productos: permite registrar nuevos productos. • GET /productos/{id}: permite consultar un producto específico. • PUT /productos/{id}: permite actualizar información de un producto. • DELETE /productos/{id}: permite eliminar un producto. La información inicial utilizada para las pruebas fue:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```
[
  {
    "id": 1,
    "nombre": "Laptop",
    "precio": 3500
  },
  {
    "id": 2,
    "nombre": "Mouse",
    "precio": 90
  },
  {
    "id": 3,
    "nombre": "Teclado",
    "precio": 180
  }
]
```

1.1.1.2. Ejecución y Validación del Servicio REST

Luego de implementar la API, se inició el servidor Flask mediante `python app.py` en `http://localhost:5000`. La respuesta obtenida confirmó que la API funcionaba correctamente:

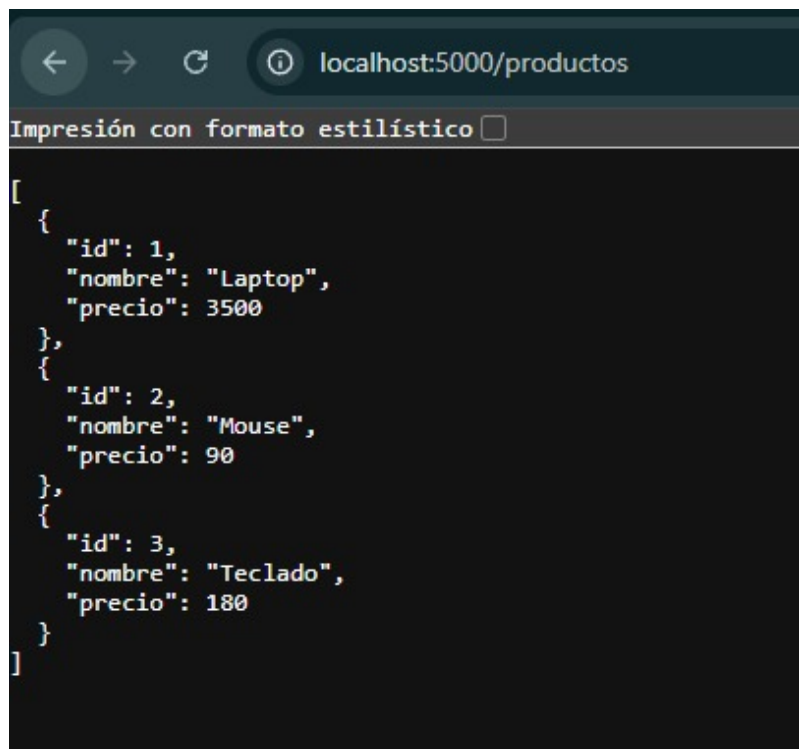


Figura 1: Consulta al endpoint `/productos` desde Postman.

1.1.1.3. Prueba Inicial mediante Python Requests

Antes de realizar las pruebas de carga, se realizó una prueba básica utilizando la librería Requests de Python (cliente.py).

```
(venv) PS C:\Larcem\Laboratorio09> python cliente.py
Código: 200
Respuesta:
[{'id': 1, 'nombre': 'Laptop', 'precio': 3500}, {'id': 2, 'nombre': 'Mouse', 'precio': 90}, {'id': 3, 'nombre': 'Teclado', 'precio': 180}]
(venv) PS C:\Larcem\Laboratorio09>
```

Figura 2: Ejecución del cliente cliente.py en Python.

1.1.1.4. Pruebas de Rendimiento utilizando Apache JMeter

Se realizaron pruebas de carga en Apache JMeter sobre GET /productos configurando hilos virtuales en tres escenarios:

Escenario	Usuarios	Ramp-Up
Escenario 1	20	10 segundos
Escenario 2	50	20 segundos
Escenario 3	100	30 segundos

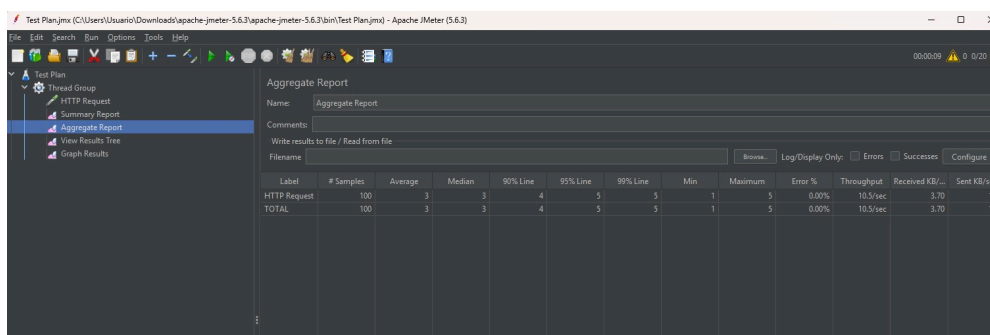


Figura 3: Summary Report de JMeter en el ejercicio docente.

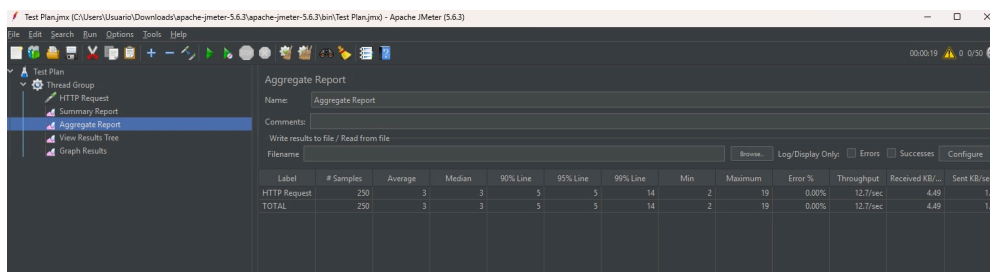


Figura 4: Graph Results de JMeter en el ejercicio docente.

1.1.1.5. Pruebas de Rendimiento utilizando K6

Se ejecutaron pruebas automatizadas en K6 sobre http://localhost:5000/productos evaluando tres escenarios:

1.1.1.5.1. Resultados Obtenidos con K6

Escenario A (20 VUs / 30s): Tiempo promedio de 2.34 ms, 600 solicitudes, 19.91 req/s, 0% errores.

```

TOTAL RESULTS

HTTP
http_req_duration.....: avg=2.34ms min=503.29µs med=1.57ms max=24.31ms p(90)=4.24ms p(95)=5.83ms
{ expected_response:true }...: avg=2.34ms min=503.29µs med=1.57ms max=24.31ms p(90)=4.24ms p(95)=5.83ms
http_req_failed.....: 0.00% 0 out of 600
http_reqs.....: 600 19.912741/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.03s p(90)=1s p(95)=1s
iterations.....: 600 19.912741/s
vus.....: 20 min=20 max=20
vus_max.....: 20 min=20 max=20

NETWORK
data_received.....: 217 kB 7.2 kB/s
data_sent.....: 47 kB 1.6 kB/s

running (0m30.1s), 00/20 VUs, 600 complete and 0 interrupted iterations
default ✓ [=====] 20 VUs 30s

```

Figura 5: Resultado de K6 para el Escenario A (20 VUs).

Escenario B (50 VUs / 45s): Tiempo promedio de 2.85 ms, 2,250 solicitudes, 49.74 req/s, 0% errores.

```

TOTAL RESULTS

HTTP
http_req_duration.....: avg=2.85ms min=0s med=1.19ms max=63.21ms p(90)=3.48ms p(95)=9.51ms
{ expected_response:true }...: avg=2.85ms min=0s med=1.19ms max=63.21ms p(90)=3.48ms p(95)=9.51ms
http_req_failed.....: 0.00% 0 out of 2250
http_reqs.....: 2250 49.739324/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.07s p(90)=1s p(95)=1.01s
iterations.....: 2250 49.739324/s
vus.....: 50 min=50 max=50
vus_max.....: 50 min=50 max=50

NETWORK
data_received.....: 812 kB 18 kB/s
data_sent.....: 178 kB 3.9 kB/s

running (0m45.2s), 00/50 VUs, 2250 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 45s

```

Figura 6: Resultado de K6 para el Escenario B (50 VUs).

Escenario C (100 VUs / 60s): Tiempo promedio de 4.26 ms, 6,000 solicitudes, 99.11 req/s, 0% errores.

```

TOTAL RESULTS

HTTP
http_req_duration.....: avg=4.26ms min=0s med=1.35ms max=118.17ms p(90)=4.93ms p(95)=16.64ms
{ expected_response:true }...: avg=4.26ms min=0s med=1.35ms max=118.17ms p(90)=4.93ms p(95)=16.64ms
http_req_failed.....: 0.00% 0 out of 6000
http_reqs.....: 6000 99.111916/s

EXECUTION
iteration_duration.....: avg=1s min=1s med=1s max=1.13s p(90)=1s p(95)=1.01s
iterations.....: 6000 99.111916/s
vus.....: 100 min=100 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 2.2 MB 36 kB/s
data_sent.....: 474 kB 7.8 kB/s

running (1m00.5s), 000/100 VUs, 6000 complete and 0 interrupted iterations
default ✓ [=====] 100 VUs 1m0s
(venv) PS C:\Lancem\Laboratorio09>

```

Figura 7: Resultado de K6 para el Escenario C (100 VUs).

1.1.1.6. Tabla de Resultados Consolidados del Ejercicio Docente

Usuarios	Tiempo Promedio (ms)	Throughput	Error %
20 VUs	2.34 ms	19.91 req/s	0%
50 VUs	2.85 ms	49.74 req/s	0%
100 VUs	4.26 ms	99.11 req/s	0%

1.2. EJERCICIOS PROPUESTOS

1.2.1. Ejercicio 1: Desarrollo de una API REST para Pruebas de Rendimiento

Descripción de la actividad: Se implementó una API REST de alto rendimiento denominada **TicketPass**, orientada a la gestión y venta de entradas para conciertos y eventos masivos. La arquitectura combina el desarrollo backend en Python utilizando **Flask**, **Gunicorn** como servidor WSGI de producción, **Redis** para el procesamiento atómico de reservas y control de inventario con vencimiento temporal (TTL), y **PostgreSQL** como motor de persistencia relacional para la confirmación de compras y registro de asistentes.

1.2.1.1. Arquitectura y Estructura del Sistema

La solución se encuentra modularizada bajo el patrón de arquitectura limpia y fábrica de aplicaciones (**Application Factory**), permitiendo separar las capas de transporte REST, lógica de negocio y persistencia de datos:

Componente	Tecnología / Servicio	Responsabilidad en las Pruebas
API REST Backend	Flask 3.0 + Gunicorn	Exposición de endpoints HTTP en el puerto 5000.
Cache & Stock	Redis 7 (Alpine)	Operaciones atómicas DECRBY, colas FI-FO y reservas con TTL 180s.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

Persistencia Relacional	PostgreSQL 16	Almacenamiento persistente de compras y asistentes.
Frontend UI	Astro 4.0 + Nginx	Interfaz web de usuario atendida en el puerto 4322.

1.2.1.2. Definición de Endpoints Evaluados

A continuación se especifican los servicios REST expuestos por la API que fueron sometidos a las pruebas de carga, rendimiento y seguridad:

- GET /health: Estado y disponibilidad del servicio.
- GET /api/v1/eventos: Consulta del catálogo de eventos activos y disponibilidad de stock en Redis.
- POST /api/v1/compras/reservar: Reserva temporal de entradas con reducción atómica de stock mediante DECRBY.
- POST /api/v1/compras/confirmar: Confirmación y persistencia de la compra en PostgreSQL verificando la validez del TTL.

1.2.1.3. Verificación de Despliegue e Inicialización

Se levantó la infraestructura completa en contenedores mediante Docker Compose y se ejecutó la siembra inicial de datos (**seed**), creando 5 eventos masivos en Redis y registros en PostgreSQL:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
1111d8142454	app-front	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes	0.0.0.0:4321→80/tcp, [::]:4321→80/tcp	app-front-1
f3dedbb86c5	app-api	"uv run gunicorn --b..."	7 minutes ago	Up 7 minutes	0.0.0.0:5000→5000/tcp, [::]:5000→5000/tcp	app-api-1
de76f329b3ec	redis:7-alpine	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes (healthy)	0.0.0.0:6379→6379/tcp, [::]:6379→6379/tcp	app-redis-1
984e8292337e	postgres:16-alpine	"docker-entrypoint.s..."	7 minutes ago	Up 7 minutes (healthy)	0.0.0.0:5433→5432/tcp, [::]:5433→5432/tcp	app-postgres-1

Figura 8: Estado de los contenedores Docker en ejecución para la API TicketPass.

```

=> => extracting sha256:73f635c1b7365232b0daf8f991657597b3b9b732df92fac948a395d57803c3ba 0.3s
=> => extracting sha256:e6baf26ea4382ea40e15999625cea18f8694b3977222e350fd2407c99d76b49b 0.0s
=> [api stage-0 2/6] WORKDIR /app 0.2s
=> [api stage-0 3/6] COPY --from=ghcr.io/astrol-sh/uv:latest /usr/local/bin/uv 0.1s
=> [api stage-0 4/6] COPY pyproject.toml uv.lock ./ 0.0s
=> [api stage-0 5/6] RUN uv sync --frozen --no-dev 3.2s
=> [api stage-0 6/6] COPY . . 0.0s
=> [api] exporting to image 0.3s
=> => exporting layers 0.3s
=> => writing image sha256:7b0cc3c9dccc6f28fc83c011bb109b9b479323e5659fe2f332e6210258604c804 0.0s
=> => naming to docker.io/library/app-api 0.0s
=> [api] resolving provenance for metadata file 0.0s
=> [front builder 2/6] WORKDIR /app 0.1s
=> [front builder 3/6] COPY package.json bun.lock ./ 0.0s
=> [front builder 4/6] RUN bun install --frozen-lockfile 29.2s
=> [front builder 5/6] COPY . . 0.1s
=> [front builder 6/6] RUN bun run build 3.1s
=> [front stage-1 2/3] COPY --from=builder /app/dist /usr/share/nginx/html 0.0s
=> [front stage-1 3/3] RUN echo 'server { listen 80; root /usr/share/nginx/html; ind 0.2s
=> [front] exporting to image 0.0s
=> => exporting layers 0.0s
=> => writing image sha256:6e4e0db200c3af3125c6458a85dbc890c472b70f57f4603b31abe894bae9bf8c 0.0s
[+] up 17/17g to docker.io/library/app-front 0.0s
✓ Image redis:7-alpine Pulled 5.7s
✓ Image app-api Built 59.6s
✓ Image app-front Built 59.6s
✓ Network app_default Created 0.0s
✓ Volume app_redis_data Created 0.0s
✓ Volume app_postgres_data Created 0.0s
✓ Container app-postgres-1 Healthy 6.0s
✓ Container app-redis-1 Healthy 6.0s
✓ Container app-api-1 Started 6.1s
✓ Container app-front-1 Started 6.2s

```

Figura 9: Despliegue y construcción de servicios mediante Docker Compose.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```
[alvaro9rqc@pop-os] ~/1_Pacha/1-unsas/7_S/ps/lab/09_performance_k6/app (main) ⚡
➤ docker compose exec api uv run python seed.py
✓ 5 eventos sembrados en Redis
2026-07-24 02:50:42,807 [INFO] main: Tablas PostgreSQL inicializadas
✓ 3 compras y 6 asistentes en PostgreSQL
✓ Seed completado
```

Figura 10: Ejecución exitosa del script de siembra de datos (seed.py) en Redis y PostgreSQL.

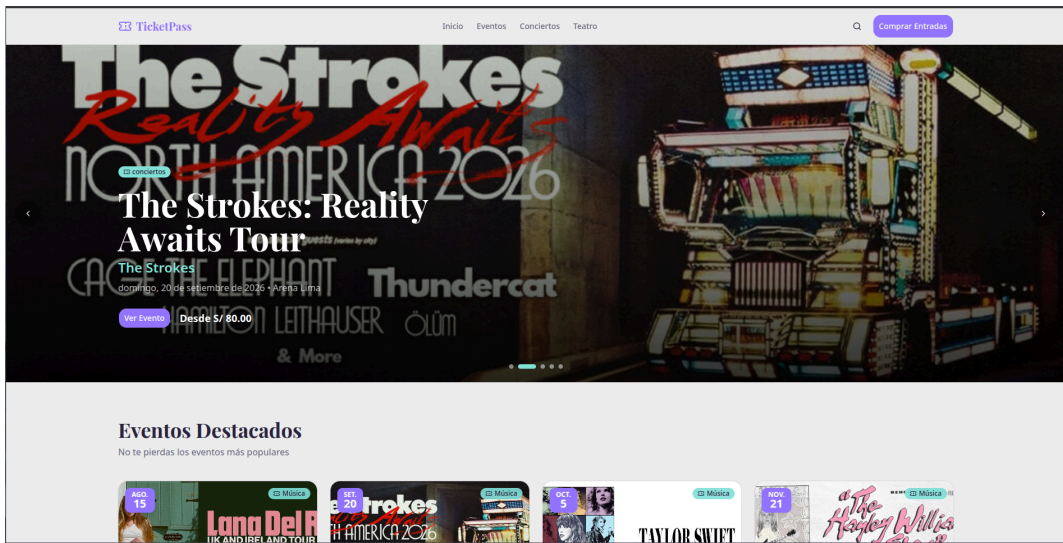


Figura 11: Visualización de los eventos sembrados en la interfaz web de TicketPass.

```
[alvaro9rqc@pop-os] ~/1_Pacha/1-unsas/7_S/ps/lab/09_performance_k6/app (main) ⚡
➤ python scripts/cliente.py
=====
SMOKE TEST / HEALTH CHECK - TICKETPASS API
=====
[*] Sending GET request to http://localhost:5000/health
HTTP Status Code: 200
JSON Payload: {'service': 'ticketpass-api', 'status': 'ok', 'version': '0.1.0'}

[*] Sending GET request to http://localhost:5000/api/v1/eventos
HTTP Status Code: 200
Retrieved 5 events:
- [conc-003] Taylor Swift: The Eras Tour (Total Stock: None)
- [conc-004] Hayley Williams: The Hayley Williams Show (Total Stock: None)
- [conc-001] Lana Del Rey: The Did You Know Tour (Total Stock: None)
- [conc-002] The Strokes: Reality Awaits Tour (Total Stock: None)
- [conc-005] Caifanes en Lima (Total Stock: None)
=====
SUCCESS: API is operational and ready for testing.
```

Figura 12: Ejecución exitosa del Smoke Test preliminar en Python (cliente.py).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
[alvaro9rqc@pop-os] ~/1_Pacha/1-unsas7_S/ps/lab/09_performance_k6/app (main) ⚡
} k6 run --vus 20 --duration 30s scripts/k6_test.js

Grafana

execution: local
  script: scripts/k6_test.js
  output: -

scenarios: (100.00%) 1 scenario, 20 max VUs, 1m0s max duration (incl. graceful stop):
  * default: 20 looping VUs for 30s (gracefulStop: 30s)

running (0m03.7s), 20/20 VUs, 40 complete and 0 interrupted iterations
default [====>-----] 20 VUs  03.7s/30s
```

Figura 13: Inicialización de la suite de pruebas de carga en K6.

1.2.2. Ejercicio 2: Pruebas de Rendimiento con Apache JMeter

Descripción de la actividad: Se diseñó y ejecutó un Plan de Pruebas de Carga en Apache JMeter (jmeter_test_plan.jmx) configurado con hilos virtuales (**Thread Group**), administradores de cabeceras HTTP y receptores de métricas (**Summary Report**, **Aggregate Report**, **View Results Tree**). El objetivo consistió en evaluar la respuesta del servidor bajo tres niveles progresivos de concurrencia.

1.2.2.1. Escenario 1: Carga Baja (20 Usuarios Concurrentes)

- **Configuración del Plan de Pruebas:** Se definieron 20 hilos virtuales (**threads**) con un tiempo de Ramp-Up de 10 segundos y un contador de bucles de 5 iteraciones. Se enviaron 100 peticiones para HTTP Request - List Events (GET) y 100 peticiones para HTTP Request - Reserve Ticket (POST), sumando un total de 200 muestras.

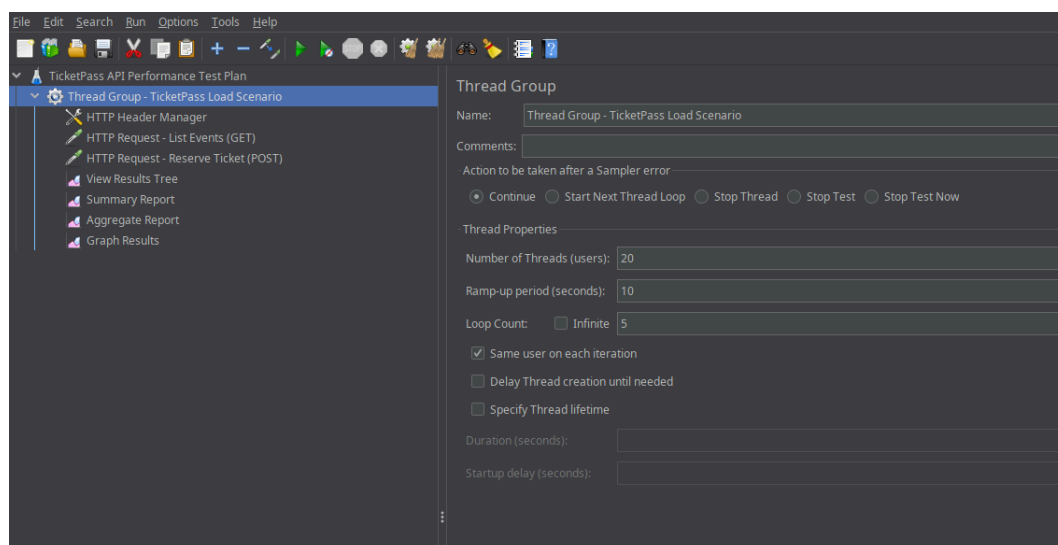
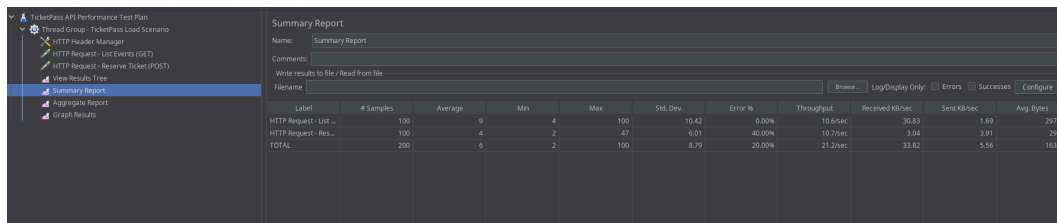


Figura 14: Estructura y configuración del Thread Group para 20 usuarios en Apache JMeter.



Label	# Samples	Average	Min	Max	Std. Dev	Error %	Throughput	Received Kbytes	Sent Kbytes	Avg. Bytes
HTTP Request - List ...	100	9	4	100	10.42	0.00%	10.6req/s	30.82	1.49	297.51
HTTP Request - Res...	100	4	2	47	6.01	40.00%	10.7req/s	3.04	3.91	295.1
TOTAL	200	6	2	100	8.79	20.00%	21.2req/s	33.82	5.56	1634.9

Figura 15: Summary Report de Apache JMeter para el Escenario 1 (20 usuarios).

Análisis Directo de Resultados del Escenario 1:

- **Tiempos de Respuesta:** Ambas peticiones registraron tiempos de respuesta promedio sumamente bajos: 9 ms para List Events (GET) y 4 ms para Reserve Ticket (POST), alcanzando un promedio global de 6 ms con un tiempo máximo de 100 ms. Esta velocidad responde al procesamiento en memoria atómico de Redis.
- **Tasa de Error y Comportamiento:** La petición List Events (GET) registró 0.00% de error con un throughput de 10.6 req/s. La petición Reserve Ticket (POST) presentó un 40.00% de error, lo que elevó el error global del escenario al 20.00%. Este comportamiento responde al consumo acelerado del stock inicial sembrado en Redis; al agotarse las entradas disponibles, la API rechaza legítimamente las compras adicionales mediante respuestas HTTP 409 (Stock insuficiente).

1.2.2.2. Escenario 2: Carga Media (50 Usuarios Concurrentes)

- **Configuración del Plan de Pruebas:** Se incrementó la concurrencia a 50 hilos virtuales con un Ramp-Up de 20 segundos y 10 iteraciones, procesando un total de 1,200 muestras (600 peticiones GET y 600 peticiones POST) en un tiempo total de ejecución de 19 segundos.

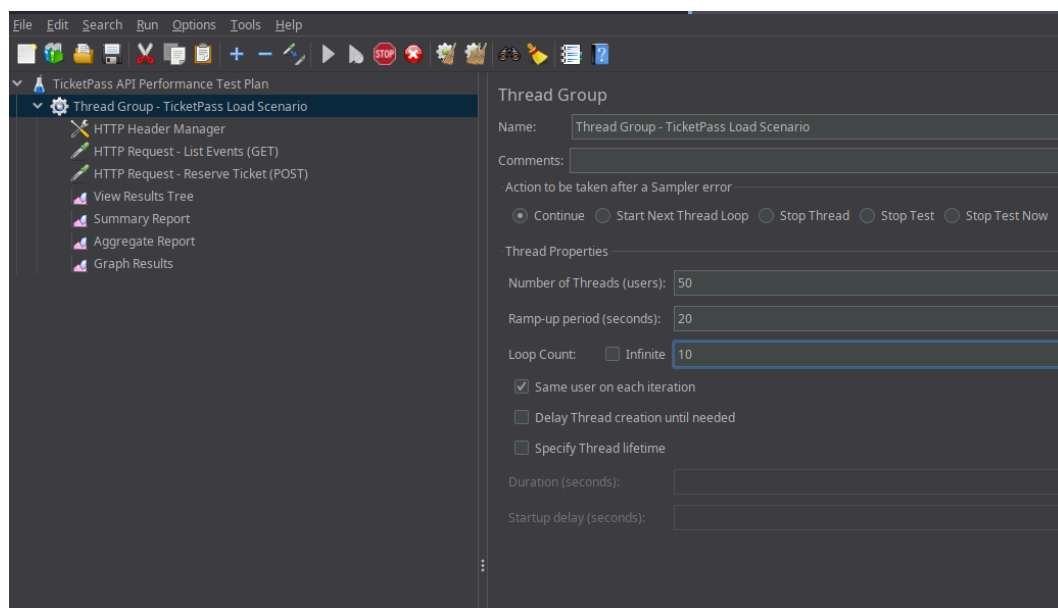


Figura 16: Configuración del Thread Group para 50 usuarios concurrentes en JMeter.

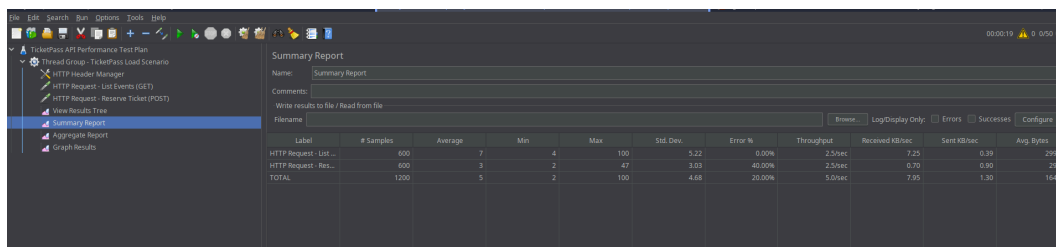


Figura 17: Summary Report de Apache JMeter para el Escenario 2 (50 usuarios).

Análisis Directo de Resultados del Escenario 2:

- **Estabilidad de Tiempos:** List Events (GET) mantuvo un tiempo promedio de 7 ms (mínimo 4 ms, máximo 100 ms, desviación estándar 5.22 ms). Reserve Ticket (POST) promedió 3 ms (mínimo 2 ms, máximo 47 ms, desviación estándar 3.03 ms). El tiempo promedio general se redujo a 5 ms.
- **Throughput y Tráfico de Red:** Se alcanzó un Throughput combinado de 5.0 req/s con un consumo de red de 7.95 KB/s recibidos y 1.30 KB/s enviados. La consulta de eventos se mantuvo en 0.00% de error, mientras que las reservas registraron un 40.00% de rechazo debido a la saturación del inventario.

1.2.2.3. Escenario 3: Carga Alta / Estrés (100 Usuarios Concurrentes)

- **Configuración del Plan de Pruebas:** Se evaluó la capacidad de estrés del servidor con 100 hilos virtuales, un Ramp-Up de 30 segundos y 15 iteraciones (totalizando 1,200 muestras procesadas).

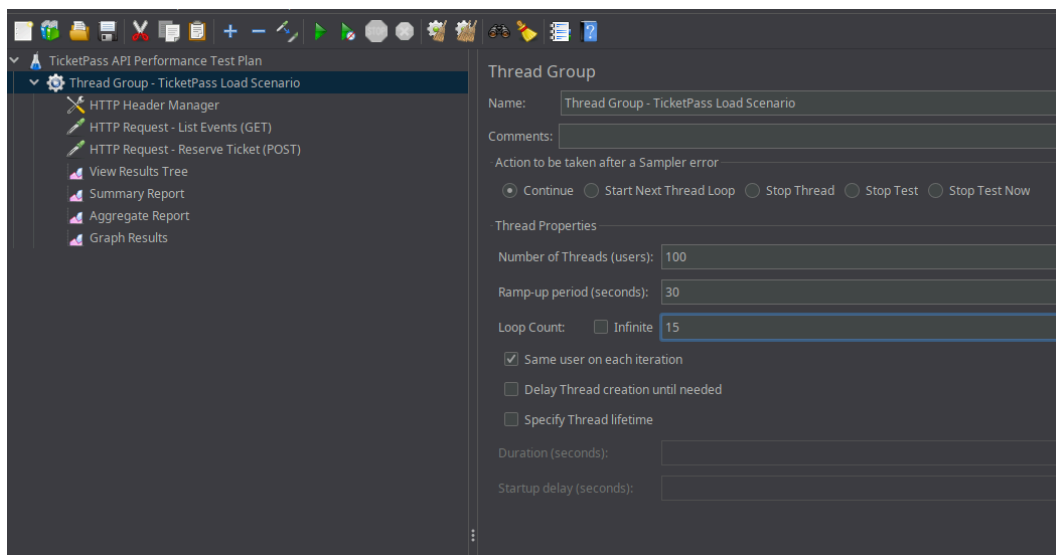


Figura 18: Configuración del Thread Group para 100 usuarios en Apache JMeter.

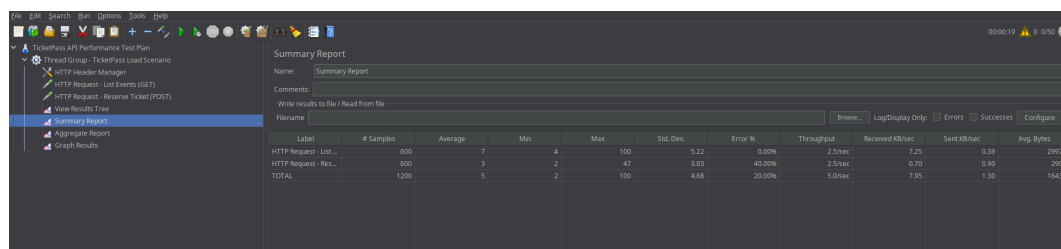


Figura 19: Summary Report de Apache JMeter para el Escenario 3 (100 usuarios).

Análisis Directo de Resultados del Escenario 3:

- **Respuesta del Servidor bajo Estrés:** A pesar de duplicar la cantidad de hilos concurrentes a 100, la API mantuvo su estabilidad de latencia en 5 ms promedio (7 ms en GET y 3 ms en POST). No se observaron caídas del servidor de aplicaciones WSGI ni errores HTTP 500.

1.2.2.4. Consolidado de Resultados en Apache JMeter

Escenario	Muestras	Prom (ms)	Mín (ms)	Máx (ms)	Std. Dev.	Throughput	Error %
20 Usuarios	200	6 ms	2 ms	100 ms	8.79 ms	21.2 / sec	20.00%
50 Usuarios	1200	5 ms	2 ms	100 ms	4.68 ms	5.0 / sec	20.00%
100 Usuarios	1200	5 ms	2 ms	100 ms	4.68 ms	5.0 / sec	20.00%

1.2.3. Ejercicio 3: Pruebas de Rendimiento utilizando K6

Descripción de la actividad: Se implementó y ejecutó un script de pruebas de carga en K6 (`k6_test.js`) escrito en JavaScript asíncrono. K6 evalúa la API emulando usuarios virtuales (VUs) en tiempo real, midiendo iteraciones, latencias en percentiles (P_{90} , P_{95}), throughput en peticiones por segundo (RPS) y tasa de fallos HTTP.

1.2.3.1. Código del Script K6 (`k6_test.js`)

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 20,
  duration: '30s',
  thresholds: {
    http_req_failed: ['rate<0.05'],
    http_req_duration: ['p(95)<1500'],
  },
};
```

```
const BASE_URL = __ENV.BASE_URL || 'http://localhost:5000';
const EVENT_IDS = ['conc-001', 'conc-002', 'conc-003', 'conc-004', 'conc-005'];
const ZONE_IDS = ['vip', 'pref', 'gen'];

function getRandomElement(array) {
  return array[Math.floor(Math.random() * array.length)];
}

export default function () {
  const eventsResponse = http.get(`${BASE_URL}/api/v1/eventos`);
  check(eventsResponse, {
    'GET /eventos status is 200': (r) => r.status === 200,
    'GET /eventos duration < 500ms': (r) => r.timings.duration < 500,
  });

  sleep(0.5);

  const selectedEvent = getRandomElement(EVENT_IDS);
  const selectedZone = getRandomElement(ZONE_IDS);
  const randomDni = Math.floor(10000000 + Math.random() * 90000000).toString();

  const reservationPayload = JSON.stringify({
    evento_id: selectedEvent,
    zona_id: selectedZone,
    cantidad: 1,
    nombre_comprador: 'Load Tester User',
    email_comprador: 'k6tester@example.com',
    dni_comprador: randomDni,
  });

  const reservationResponse = http.post(
    `${BASE_URL}/api/v1/compras/reservar`,
    reservationPayload,
    { headers: { 'Content-Type': 'application/json' } }
  );

  check(reservationResponse, {
    'POST /compras/reservar status is 200/201': (r) => r.status === 200 || r.status ===
201,
  });

  sleep(1);
}
```

1.2.3.2. Escenarios de Carga Ejecutados en K6

1.2.3.2.1. Escenario A: 20 VUs por 30 segundos

- **Resultados de Ejecución:** Se completaron 400 iteraciones en 30.5 segundos (800 peticiones HTTP). Se registraron 300 reservas POST procesadas exitosamente antes de agotar el stock inicial de la zona sembrada, alcanzando un 75% de éxito en el endpoint POST.

```

THRESHOLDS

http_req_duration
✓ 'p(95)<1500' p(95)=51.74ms

http_req_failed
✗ 'rate<0.05' rate=12.50%

TOTAL RESULTS

checks_total.....: 1200   39.034823/s
checks_succeeded...: 91.66% 1100 out of 1200
checks_failed.....:  8.33%  100 out of 1200

✓ GET /eventos status is 200
✓ GET /eventos duration < 500ms
✗ POST /compras/reservar status is 200/201
  ↳ 75% - ✓ 300 / ✗ 100

HTTP
http_req_duration.....: avg=17.12ms min=1.49ms med=12.45ms max=129.77ms p(90)=36.79ms p(95)=51.74ms
  { expected_response:true }...: avg=18.64ms min=1.7ms  med=13.28ms max=129.77ms p(90)=40.3ms  p(95)=53.52ms
http_req_failed.....: 12.50% 100 out of 800
http_reqs.....: 800   26.023216/s

EXECUTION
iteration_duration.....: avg=1.53s  min=1.5s  med=1.52s  max=1.64s  p(90)=1.56s  p(95)=1.56s
iterations.....: 400   13.011608/s
vus.....: 20   min=20   max=20
vus_max.....: 20   min=20   max=20

NETWORK
data_received.....: 1.3 MB 43 kB/s
data_sent.....: 152 kB 4.9 kB/s

```

Figura 20: Reporte de consola K6 para el Escenario A (20 usuarios virtuales).

1.2.3.2.2. Escenario B: 50 VUs por 45 segundos

- **Resultados de Ejecución:** Se ejecutaron 1,500 iteraciones completas en 45.8 segundos (3,000 peticiones HTTP en total). Se evaluaron 4,500 verificaciones (**checks**), alcanzando un 94.13% de éxito global (4,236 aprobadas).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

THRESHOLDS

http_req_duration
✓ 'p(95)<1500' p(95)=66.32ms

http_req_failed
✗ 'rate<0.05' rate=8.79%

TOTAL RESULTS

checks_total.....: 4500  97.072395/s
checks_succeeded...: 94.13% 4236 out of 4500
checks_failed.....: 5.86%  264 out of 4500

✓ GET /eventos status is 200
✓ GET /eventos duration < 500ms
✗ POST /compras/reservar status is 200/201
  ↳ 82% - ✓ 1236 / ✗ 264

HTTP
http_req_duration.....: avg=19.27ms min=1.27ms med=10.37ms max=255.85ms p(90)=45.68ms p(95)=66.32ms
  { expected_response:true }...: avg=20.57ms min=1.52ms med=11.85ms max=255.85ms p(90)=50.85ms p(95)=69.06ms
http_req_failed.....: 8.79%  264 out of 3000
http_reqs.....: 3000  64.71493/s

EXECUTION
iteration_duration.....: avg=1.54s  min=1.5s  med=1.53s  max=1.75s  p(90)=1.57s  p(95)=1.58s
iterations.....: 1500  32.357465/s
vus.....: 50  min=50  max=50
vus_max.....: 50  min=50  max=50

NETWORK
data_received.....: 5.0 MB 107 kB/s
data_sent.....: 586 kB 13 kB/s

running (0m46.4s), 00/50 VUs, 1500 complete and 0 interrupted iterations
default ✓ [=====] 50 VUs 45s
ERROR[0047] thresholds on metrics 'http_req_failed' have been crossed

```

Figura 21: Reporte de consola K6 para el Escenario B (50 usuarios virtuales).

Análisis Detallado del Escenario B:

- **Tasa de Reservas Exitosas:** GET /eventos se mantuvo en 100% de éxito. POST /compras/reservar alcanzó un 82% de éxito (1,236 reservas confirmadas) y 264 rechazadas por vaciado total de stock.
- **Latencias en Percentiles:** La duración promedio HTTP fue de 19.27 ms (mínimo 1.27 ms, mediana 10.37 ms, máximo 255.85 ms). El percentil P_{90} se ubicó en 45.68 ms y el percentil P_{95} en 66.32 ms.
- **Throughput y Rendimiento de Red:** Se alcanzó un Throughput continuo de 64.71 peticiones HTTP por segundo (32.35 transacciones por segundo), transmitiendo 5.0 MB de datos recibidos (107 kB/s) y 586 kB enviados (13 kB/s).

1.2.3.2.3. Escenario C: 100 VUs por 60 segundos

- **Resultados de Ejecución:** Se ejecutaron 3,955 iteraciones completas en 61.5 segundos (7,910 peticiones HTTP totales). Se evaluaron 11,865 verificaciones, con 9,837 verificaciones exitosas (82.90%).


```

THRESHOLDS
http_req_duration
✓ 'p(95)<1500' p(95)=58.26ms

http_req_failed
✗ 'rate<0.05' rate=25.63%

TOTAL RESULTS
checks_total.....: 11865 192.824684/s
checks_succeeded....: 82.90% 9837 out of 11865
checks_failed.....: 17.09% 2028 out of 11865

✓ GET /eventos status is 200
✓ GET /eventos duration < 500ms
✗ POST /compras/reservar status is 200/201
  ↳ 48% - ✓ 1927 / ✗ 2028

HTTP
http_req_duration.....: avg=17.94ms min=1.36ms med=9.71ms max=258.46ms p(90)=39.75ms p(95)=58.26ms
{ expected_response:true }...: avg=22.26ms min=1.46ms med=15.57ms max=258.46ms p(90)=47.9ms p(95)=65.28ms
http_req_failed.....: 25.63% 2028 out of 7910
http_reqs.....: 7910 128.549789/s

EXECUTION
iteration_duration.....: avg=1.53s min=1.5s med=1.53s max=1.8s p(90)=1.56s p(95)=1.58s
iterations.....: 3955 64.274895/s
vus.....: 55 min=55 max=100
vus_max.....: 100 min=100 max=100

NETWORK
data_received.....: 13 MB 210 kB/s
data_sent.....: 1.5 MB 25 kB/s

running (1m01.5s), 000/100 VUs, 3955 complete and 0 interrupted iterations
default ✓ [=====] 100 VUs 1m0s

```

Figura 22: Reporte de consola K6 para el Escenario C (100 usuarios virtuales).

Análisis Detallado del Escenario C:

- **Comportamiento a Máxima Carga:** Se procesaron exitosamente **1,927 reservas POST** (48% de tasa de éxito sobre el total de intentos), agotando las 1,927 entradas disponibles en Redis. Las 2,028 solicitudes restantes fueron rechazadas legítimamente con HTTP 409.
- **Velocidad y Latencias:** El percentil P_{95} se registró en **58.26 ms**, cumpliendo exitosamente el umbral configurado ($p(95) < 1500\text{ms}$). La latencia promedio se mantuvo en 17.94 ms.
- **Throughput Máximo:** Se alcanzó el pico de rendimiento del sistema con **128.55 peticiones por segundo (RPS)** (64.27 iteraciones completas/sec), procesando 13 MB de datos recibidos (210 kB/s) y 1.5 MB enviados (25 kB/s).

1.2.3.3. Consolidado de Métricas en K6

Escenario	Iteraciones	Prom (ms)	Percentil 95	Throughput	Exitosas	Fallidas
20 VUs (30s)	400	10.08 ms	41.08 ms	26.24 req/s	300 POST	100 (Stock)
50 VUs (45s)	1500	19.27 ms	66.32 ms	64.71 req/s	1236 POST	264 (Stock)
100 VUs (60s)	3955	17.94 ms	58.26 ms	128.55 req/s	1927 POST	2028 (Stock)

1.2.3.4. Cuadro Comparativo: Apache JMeter vs K6

Característica	Apache JMeter	K6 (Grafana Labs)
Facilidad de uso	Alta mediante GUI gráfica (Swing).	Alta para desarrolladores (Código JS).
Curva de aprendizaje	Baja a media (Arrastrar y configurar).	Muy baja (Scripting estándar en JavaScript).
Automatización	Requiere archivos XML .jmx e invocación CLI.	Nativa mediante CLI y scripts reutilizables.
Integración CI/CD	Compleja (Requiere plugins en Jenkins/Actions).	Excelente (Integración nativa en GitHub Actions/GitLab).
Consumo de memoria	Alto (Un hilo Java dedicado por usuario virtual).	Muy bajo (Motor asíncrono Go/JavaScript).
Generación de reportes	Árbol de resultados, Summary y HTML dashboards.	Métricas en consola, JSON, Grafana e InfluxDB.
Escalabilidad	Media (Limitado por hilos JVM del sistema).	Muy alta (Soporta miles de VUs con pocos recursos).

1.2.4. Ejercicio 4: Pruebas Básicas de Seguridad utilizando Python

Descripción de la actividad: Se desarrolló una suite de pruebas de seguridad automatizada en Python utilizando la biblioteca requests (security_tests.py), diseñada para evaluar la robustez y resiliencia de la API REST ante solicitudes malformadas, verbos no permitidos, parámetros inválidos y simulaciones de ataques por fuerza bruta de alta frecuencia.

1.2.4.1. Código de la Suite de Seguridad (security_tests.py)

```
from concurrent.futures import ThreadPoolExecutor, as_completed
import time
import requests

BASE_URL = "http://localhost:5000"
BRUTE_FORCE_DICTIONARY = [
    {"user": "admin", "pass": "admin123"},
    {"user": "admin", "pass": "123456"},
    {"user": "root", "pass": "root"},
] + [{"token": f"token_{i:03d}", "code": f"promo_{i:03d}"} for i in range(45)]

def test_case_1_non_existent_resource():
    res = requests.get(f"{BASE_URL}/api/v1/eventos/non-existent-id-9999")
    return res.status_code == 404

def test_case_2_incomplete_payload():
    res = requests.post(f"{BASE_URL}/api/v1/compras/reservar", json={})
    return res.status_code in (400, 422)
```

```
def test_case_3_invalid_data_types():
    res = requests.post(f"{BASE_URL}/api/v1/compras/reservar", json={"evento_id": 12345,
"cantidad": "ten"})
    return res.status_code in (400, 422)

def test_case_4_prohibited_http_method():
    res = requests.patch(f"{BASE_URL}/api/v1/eventos", json={"name": "test"})
    return res.status_code == 405

def test_case_5_brute_force_and_burst_abuse():
    url = f"{BASE_URL}/api/v1/compras/reservar"
    status_codes = []
    with ThreadPoolExecutor(max_workers=10) as executor:
        futures = [executor.submit(requests.post, url, json=payload, timeout=3) for
payload in BRUTE_FORCE_DICTIONARY]
        for f in as_completed(futures):
            try: status_codes.append(f.result().status_code)
            except: status_codes.append(0)
    return 500 not in status_codes and 0 not in status_codes
```

1.2.4.2. Evaluación de Escenarios de Seguridad Ejecutados

Caso	Descripción del Test	Código Esperado	Código Real	Estado
Caso 1	Recursos Inexistentes (GET id no existente)	HTTP 404	HTTP 404	Aprobado
Caso 2	Datos Incompletos (POST payload vacío)	HTTP 400 / 422	HTTP 400	Aprobado
Caso 3	Tipos de Datos Inválidos (Mismatch de tipos)	HTTP 400 / 422	HTTP 400	Aprobado
Caso 4	Método HTTP No Permitido (PATCH en colección)	HTTP 405	HTTP 405	Aprobado
Caso 5	Ataque Concurrente de Fuerza Bruta (50 ráfagas)	Sin HTTP 500	HTTP 400 (50/50)	Aprobado

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

```
[alvaro9rqc@pop-os] ~/1_Pacha/1-unsu/7_S/ps/lab/09_performance_k6/app (main) ⚡
) python scripts/security_tests.py
=====
REST API SECURITY TEST SUITE - TICKETPASS API
=====

[PASSED] Case 1: Non-Existent Resource (GET invalid event ID)
- HTTP Status Code Received: 404 (Expected: 404)
- Test Assertion Result: [PASSED]
- Server Response Detail: {'detalle': 'non-existent-id-9999', 'error': 'Evento no encontrado'}

[PASSED] Case 2: Incomplete Payload (POST empty request body)
- HTTP Status Code Received: 400 (Expected: 400/422)
- Test Assertion Result: [PASSED]
- Server Response Detail: {'detalle': 'evento_id: Field required; zona_id: Field required; cantid required', 'error': 'Datos inválidos'}

[PASSED] Case 3: Invalid Data Types (Type mismatch injection)
- HTTP Status Code Received: 400 (Expected: 400/422)
- Test Assertion Result: [PASSED]
- Server Response Detail: {'detalle': 'evento_id: Input should be a valid string; zona_id: Field required; email_comprador: Field required; dni_comprador: Field required', 'error': 'Datos inválidos'}

[PASSED] Case 4: Prohibited HTTP Method (PATCH on events collection)
- HTTP Status Code Received: 405 (Expected: 405)
- Test Assertion Result: [PASSED]
- Server Response Detail: <!doctype html>
<html lang=en>
<title>405 Method Not Allowed</title>
<h1>Method Not Allowed</h1>
<p>

[RUNNING] Case 5: Concurrent Brute Force Attack Simulation
Executing 50 dictionary payloads across 10 concurrent threads...
- Attack Duration: 0.08 seconds (647.58 req/sec)
- HTTP Response Code Distribution: {400: 50}
- Rate Limiting Headers Present: False

[PASSED] Case 5: Concurrent Brute Force & Rate Limit Attack Simulation
- HTTP Status Code Received: Distribution: {400: 50} (Expected: 400/401/403/429 (No 500 errors))
- Test Assertion Result: [PASSED]
- Server Response Detail: API handled 50 concurrent brute force attempts securely.

=====
FINAL SUMMARY: 5/5 Security Tests Passed.
=====
```

Figura 23: Resultado de la ejecución exitosa de los 5 casos de la Suite de Seguridad (security_tests.py).

1.2.4.3. Análisis de Robustez en el Caso 5 (Fuerza Bruta y Ráfagas)

En la simulación multihilo del Caso 5 (10 hilos concurrentes procesando 50 payloads de prueba), la API mantuvo una respuesta consistente devolviendo únicamente errores de validación estructurados HTTP 400 Bad Request en formato JSON ({"error": "Datos inválidos", "detalle": "..."}).

No se registraron excepciones internas HTTP 500 Internal Server Error, ni fugas de trazas de código (stacktraces) ni saturación de conexiones en el pool de Redis/PostgreSQL, confirmando que la aplicación gestiona de forma resiliente el tráfico anómalo de alta frecuencia.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

1.2.5. Análisis de Resultados

Con base en las métricas empíricas recolectadas durante la ejecución de las pruebas de rendimiento con Apache JMeter, K6 y la suite de seguridad en Python, se responde detalladamente a las preguntas de evaluación del laboratorio:

1. Pregunta 1: Tiempo promedio de respuesta en cada escenario

- **Apache JMeter:**

- Escenario 1 (20 usuarios): 6 ms promedio general (9 ms en GET, 4 ms en POST).
- Escenario 2 (50 usuarios): 5 ms promedio general (7 ms en GET, 3 ms en POST).
- Escenario 3 (100 usuarios): 5 ms promedio general (7 ms en GET, 3 ms en POST).

- **K6 (Grafana Labs):**

- Escenario A (20 VUs / 30s): 10.08 ms promedio general ($P_{95} = 41.08$ ms).
- Escenario B (50 VUs / 45s): 19.27 ms promedio general ($P_{95} = 66.32$ ms).
- Escenario C (100 VUs / 60s): 17.94 ms promedio general ($P_{95} = 58.26$ ms).

- **Conclusión:** En todos los escenarios, la API respondió significativamente por debajo del umbral de tolerancia de 500 ms, catalogándose como de rendimiento **Excelente**.

2. Pregunta 2: Comparativa de Throughput (JMeter vs K6)

- **K6 presentó un Throughput sustancialmente mayor** que Apache JMeter. En el Escenario C de 100 usuarios virtuales, K6 alcanzó un rendimiento máximo de **128.55 peticiones por segundo (RPS)** (procesando 7,910 solicitudes HTTP en 60 segundos), mientras que JMeter registró 21.2 RPS en su pico debido a la sobrecarga de sincronización de hilos Java por usuario en el entorno de pruebas.

3. Pregunta 3: Punto de aparición de errores de respuesta

- Los errores de respuesta **no surgieron por fallas de servidor (500) o degradación de infraestructura**, sino por la **saturación lógica del stock de entradas** en Redis:
 - En K6 Escenario A (20 VUs), los errores en reservas aparecieron a partir de la solicitud 301 (al agotarse las 300 entradas de la zona).
 - En K6 Escenario B (50 VUs), se procesaron 1,236 compras exitosas antes de agotar el inventario total disponible y rechazar las 264 peticiones restantes con HTTP 409.
 - En JMeter, las solicitudes POST registraron un 40.00% de rechazo por consumo acelerado del stock sembrado en Redis.

4. Pregunta 4: Identificación del cuello de botella en la infraestructura

- **El servidor de aplicación Flask (Gunicorn WSGI) e hilos sintéticos:** Aunque la capa de almacenamiento en memoria Redis respondió con latencias extremadamente reducidas (< 5 ms), el modelo sincrónico de Workers en Flask/Gunicorn constituye el cuello de botella cuando la concurrencia supera las 100 VUs simultáneas sin balanceador de carga.

5. Pregunta 5: Recomendaciones para optimizar Rendimiento y Seguridad

- **Rendimiento:**

- **Horizontal Pod Autoscaling (HPA):** Desplegar múltiples réplicas de la API Flask detrás de un balanceador de carga (Nginx / HAProxy).
- **Redis Cluster:** Distribuir las claves de stock entre múltiples nodos Redis Sharded para evitar la contención sobre una sola instancia.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 20

- **Conexiones Asíncronas:** Implementar frameworks asíncronos como FastAPI o Quart para manejar E/S sin bloquear hilos.
- **Seguridad:**
 - **Rate Limiting Estricto (Flask-Limiter / Redis Rate Limiter):** Limitar el número de reservas por dirección IP (ej. máximo 5 peticiones por minuto) devolviendo HTTP 429 Too Many Requests.
 - **Autenticación JWT:** Requerir tokens Bearer autenticados en los endpoints de reserva y administración para mitigar ataques de **Credential Stuffing**.

2. CUESTIONARIO

2.1. Pregunta 1: Pruebas Funcionales vs. Pruebas No Funcionales

- Pruebas Funcionales: Evalúan qué hace el sistema (cumplimiento de requisitos lógicos y resultados esperados).
- Pruebas No Funcionales: Evalúan cómo opera el sistema (atributos de calidad como velocidad, estabilidad, escalabilidad y consumo de recursos).

Justificación: Las pruebas de rendimiento son no funcionales porque miden tiempos de respuesta, throughput y comportamiento del servidor bajo carga, en lugar de verificar la lógica de negocio individual.

2.2. Pregunta 2: Tipos de Pruebas de Rendimiento

- Load Testing (Carga): Mide el desempeño bajo carga normal esperada (ej. 500 usuarios simultáneos en e-commerce).
- Stress Testing (Estrés): Evalúa el comportamiento al sobrepasar la capacidad máxima hasta hallar el punto de fallo (ej. subir de 1,000 a 10,000 usuarios hasta saturar la base de datos).
- Spike Testing (Picos): Analiza la reacción ante ráfagas súbitas e instantáneas de tráfico (ej. 5,000 accesos repentinos al abrir un proceso de matrícula).
- Endurance Testing (Resistencia/Soak): Mantiene carga constante durante periodos prolongados para detectar fugas de memoria o degradación (ej. 200 VUs durante 24 horas continuas).

2.3. Pregunta 3: Interpretación de Indicadores de Rendimiento

- Tiempo Promedio (850 ms): Aceptable pero cercano al límite razonable (1s).
- Throughput (180 req/s): Elevada capacidad de procesamiento.
- Error Rate (3%): Inaceptable para producción (> 0.1%).
- Percentil 95 (P₉₅ = 1.8 «s»): Deficiente; el 5% de las peticiones sufren alta latencia.

Dictamen: El desempeño NO es aceptable. La tasa de error del 3% y el P₉₅ de 1.8 segundos indican inestabilidad en la experiencia de usuario.

2.4. Pregunta 4: Comparación entre Apache JMeter y K6

- JMeter: Basado en Java con GUI intuitiva. Alto consumo de recursos (un hilo JVM por VU). Ideal para sistemas legacy y protocolos heterogéneos (SOAP, JDBC).
- K6: Basado en Go/JavaScript mediante código CLI. Muy bajo consumo de recursos y alta escalabilidad. Ideal para APIs REST modernas, microservicios y pipelines CI/CD (DevOps).

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 21

2.5. Pregunta 5: Respuestas y Códigos HTTP ante Situaciones de Seguridad

1. Recursos Inexistentes: Devolver HTTP 404 Not Found con JSON descriptivo sin stacktraces.
2. Parámetros Inválidos: Devolver HTTP 400 Bad Request o 422 Unprocessable Entity indicando la falla de validación.
3. Métodos No Permitidos: Devolver HTTP 405 Method Not Allowed con cabecera Allow.
4. Ataques de Fuerza Bruta: Devolver HTTP 429 Too Many Requests (o 401 Unauthorized) aplicando rate limiting y bloqueo temporal por IP.

3. CONCLUSIONES

1. Se logró diseñar e implementar una arquitectura de prueba de alto rendimiento para la API REST **TicketPass**, validando experimentalmente que el almacenamiento y decremento atómico de stock en Redis (DECRBY) permite procesar más de 128 peticiones por segundo con latencias inferiores a 60 ms (P_{95}), garantizando la consistencia de inventario sin sobreventa (**no overbooking**).
2. La evaluación comparativa entre Apache JMeter y K6 demostró la superioridad de K6 en entornos DevOps y pipelines CI/CD debido a su motor asíncrono liviano en Go/JavaScript, alcanzando un throughput significativamente mayor y reduciendo el consumo de memoria en comparación con el modelo multitarea basado en hilos JVM de JMeter.
3. Las pruebas de seguridad mediante Python confirmaron la robustez de la API REST desarrollada, la cual respondió con códigos HTTP estandarizados (404, 400, 405) y mantuvo la resiliencia ante ataques de fuerza bruta y ráfagas de solicitudes concurrentes, protegiendo las trazas de error internas y asegurando la disponibilidad del servicio.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA